FACULTY OF SCIENCES OF SFAX — UNIVERSITY OF SFAX

A First QLoRA Fine-Tuning Experiment for Infrastructure-as-Code Security Remediation

A preliminary experiment on Gemma-2-2B-IT

Ahmedou Yahye Kheyri

Supervised by: **Prof. Hala Bezine**

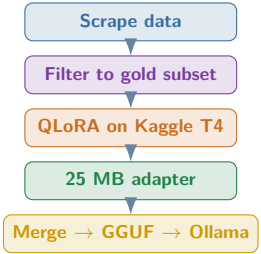
Master MRSI — PFE 2025–2026

Progress Report — April 2026

Gemma-2 · QLoRA · IaC Security · Kaggle

Outline

1. Positioning
2. Background: LoRA, QLoRA, SFT
3. Dataset
4. Method
5. Execution on Kaggle
6. Results
7. Discussion
8. Next Steps
9. Conclusion



Where this work fits in the project

Agentic IaC security framework (Rapport 2)

Seven modules cooperating under a central orchestrator:

- Contextual Analyzer
- Knowledge Base + RAG Retriever
- **Fix Generator** → this work
- External Validator (Checkov, tfsec, KICS)
- Patch Formatter & Explanation

Two informal observations on the baseline

- Off-the-shelf small LLMs (Gemma-3-4B, Gemma-4-e2B) produce plausible fixes but do not always target the declared CWE.
- On CPU, Gemma-4-e2B $\approx$ **320 s / fix** $\Rightarrow$ full eval with self-consistency is impractical on current hardware.

Why touch the Fix Generator now?

It is the *only* module whose quality depends on the underlying LLM rather than on deterministic software.

This report

A first, preliminary attempt to fine-tune a small open-weight model on our own scraped data.

Scope of this experiment

What this is

A **first, exploratory** run to validate the pipeline end-to-end.

In scope

- Small base model (≈ 2 B params)
- Subset of the scraped data (3,227 train)
- 1 epoch, 1 random seed
- 1 prompt template
- Qualitative check on $\mathbf{n = 3}$ samples
- Kaggle free T4 GPU

Out of scope

- No hyperparameter search
- No comparison against baselines
- No downstream task-level metric (scanner pass-rate)
- No multi-seed variance estimate
- No claim of competitive performance

Goal: demonstrate that the pipeline works and surface the obstacles that block a scaled experiment.

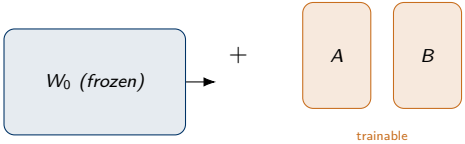
LoRA in one slide

Core idea

Freeze the pretrained weights $W_0 \in \mathbb{R}^{d \times k}$ and learn a low-rank update

$$\Delta W = BA, \quad B \in \mathbb{R}^{d \times r},\ A \in \mathbb{R}^{r \times k},\ r \ll \min(d, k).$$

Effective weight at inference: $W = W_0 + \Delta W$.



For Gemma-2-2B, $r = 16$ on attention only

- Trainable params: **9.27 M** out of 2.6 B ($\approx 0.35\,\%$)
- Adapter file: **25 MB** (vs. 5 GB full model)

QLoRA and supervised fine-tuning

QLoRA = LoRA on 4-bit base

- Base weights stored in **NF4** (4-bit NormalFloat) with double-quantised scales
- Tiles decompressed on the fly for forward pass
- Activations in fp16, gradients routed *only* through A/B
- Memory footprint for 2-B model: ≈ 5 GB fp16 $\rightarrow$ $\approx$ **1.6 GB NF4**

Consequence

A 2-B model becomes trainable on a **single 16 GB T4** — exactly what Kaggle's free tier offers.

Supervised fine-tuning (SFT) for code repair

Task: given a *detected smell* and an *insecure snippet*, output a *fixed snippet*.

- Standard causal-LM cross-entropy loss
- Loss computed **only on the model turn** (the fix tokens), prompt tokens are masked
- Closer to code-to-code translation than to free code generation

Why SFT (and not RL / preference optimisation) here?

We have before/after pairs, not ranked preferences. SFT matches the data we already have.

Dataset origin: our own scraper

Five complementary sources

- GitHub **commit search** with date-window partitioning (to bypass the 1,000-result cap)
- **GHArchive** hourly event dumps (bypasses the Search API quota)
- **Scanner test suites** (Checkov / KICS / tfsec): paired "failed" / "passed" examples
- **Known vulnerable repos**: TerraGoat, Kubernetes-Goat, WrongSecrets
- **Scanner-validated labels**: Checkov + tfsec + KICS rerun on every record

Dataset v1 — headline figures

Total records (dedup)	31,748
With a fix	16,362
Scanner-validated	10,791
Source repositories	$>2{,}000$
Smell types	18

Tool mix (v1 raw)

Terraform 50 % ▪ Kubernetes 28 % ▪ Ansible 13 % ▪ Docker 7 % ▪ CloudFormation 1.5 %

From 31,748 raw records to a 3,227-record training set

Filter	Remaining	Share of input
All validated records	31,748	100.0 %
`has_fix` = True	16,362	51.5 %
Scanner-validated only	∼10,791	34.0 %
Non-empty `code_before` and `code_after`	∼10,500	—
Combined length < 8,000 chars	∼5,500	—
Non-noop fix (`before` ≠ `after`)	**4,050**	12.76 %
Pre-existing train split (80 %)	**3,227**	—

Gold subset — better-balanced tool mix

Docker 32 % ▪ Terraform 30 % ▪ Kubernetes 30 % ▪ CloudFormation 5 % ▪ Ansible 3 %

Caveat — this is a deliberate subset

We chose the **strictest** filter for this first run. A larger (but noisier) corpus of up to ∼ 10,791 records is available and has not been used yet.

Training sample format

Slim JSON record

```
{
  "id": "a1b2c3d4",
  "iac_tool": "terraform",
  "smell_types": ["hardcoded_credential", "overly_permissive_cidr"],
  "cwes": ["CWE-798", "CWE-732"],
  "code_before": "resource aws_s3_bucket ...",
  "code_after":  "resource aws_s3_bucket ...",
  "split": "train"
}
```

Rendered at training time via the Gemma-2 chat template

```
You are a security expert specializing in Infrastructure as Code (IaC).
Fix the following {iac_tool} script. Detected smells: {smells}.
Return ONLY the corrected script, no explanation.

‘‘‘
{code_before}
‘‘‘
```

Base model: Gemma-2-2B-IT

Model	Size	Gated	Remark
google/codegemma-2b-it	2 B	Yes	Code-specialised
google/gemma-2-2b-it	2 B	Yes	**Chosen** — matches Ollama stack
Qwen/Qwen2.5-Coder-3B-Instruct	3 B	No	Stronger code model, non-gated
bigcode/starcoder2-3b	3 B	No	No chat template

Reasons for this choice

- Same family as our local Ollama stack (Gemma-3-4B, Gemma-4-e2B) — no new architecture
- Published chat template that maps onto our instruction/response pair
- Smallest Gemma-2: VRAM headroom on T4, a few hours per run

This is a pragmatic choice, not an optimum

No systematic model comparison is in scope here. Larger candidates (3 B, 7 B, 9 B) are deferred to a later iteration.

QLoRA configuration

Parameter	Value
Quantisation	4-bit NF4 (double-quant, fp16 compute)
LoRA rank r	16
LoRA α	32
LoRA dropout	0.05
Target modules	`q_proj`, `k_proj`, `v_proj`, `o_proj`
Trainable params	9,273,344 (0.35 %)
Learning rate	2×10^{-4}
Scheduler / warmup	Cosine / 3 %
Effective batch	8 (1×8 grad. accumulation)
Epochs	**1**
Optimiser	Paged AdamW 8-bit
Max seq. length	2,048 tokens
Precision	fp16 (bf16 unavailable on T4)
Gradient checkpointing	Enabled (non-reentrant)
Random seed	42

Values copied from published recipes — no hyperparameter search was performed in this first run.

Kaggle environment

Runtime

- Accelerator: NVIDIA **T4 (16 GB)**
- CUDA: 12.8
- Python: 3.12
- Session limit: 9 h per notebook
- Internet: on (to fetch base model)
- Attachments: 2 custom Kaggle datasets (scripts + filtered corpus)

Three issues encountered (for reproducibility)

- **403 Forbidden** on Gemma-2
  → accept license on HF page
- `bitsandbytes` 0.43.3 lacks CUDA 12.8 binary
  → upgrade to 0.49.2 in-place
- Kernel restart wipes env vars
  → re-run the HF-token cell first

The five Kaggle cells (summary)

Cell 1 — Install deps

```
!pip install -q -U transformers==4.44.2 \
  datasets==2.21.0 peft==0.12.0 \
  accelerate==0.34.2 bitsandbytes==0.43.3 \
  trl==0.10.1
!pip install -q -U bitsandbytes
```

Cell 2 — Expose HF token

```
from kaggle_secrets import UserSecretsClient
os.environ["HF_TOKEN"] = \
  UserSecretsClient().get_secret("HF_TOKEN")
```

Cell 3 — Smoke test (200 samples)

```
!python 02_train_qlora.py \
  --model google/gemma-2-2b-it \
```

Cell 4 — Full training (1 epoch)

```
!python 02_train_qlora.py \
  --model google/gemma-2-2b-it \
  --data-dir .../data \
  --output-dir /kaggle/working/lora_out \
  --epochs 1 --batch-size 1 \
  --grad-accum 8 --lr 2e-4
```

Cell 5 — Inference check

```
!python 03_inference.py \
  --model google/gemma-2-2b-it \
  --adapter /kaggle/working/lora_out \
  --input-file .../test.jsonl \
  --n-samples 3
```

Smoke test is a sanity check, not a curve

Produces a single final loss, not a learning curve; only the full run

Training metrics (single run, seed 42)

Metric	Value
Optimisation steps	403
Training wall-clock	1 h 47 min
Evaluation wall-clock	218 s
Average step time	15.94 s
Training throughput	0.502 samples/s
Final training loss	**0.528**
Final evaluation loss	**0.467**
Adapter size (safetensors)	25.6 MB

How to read these numbers

- **One run, one seed** — no confidence interval.
- Absolute loss is tokeniser-dependent; not directly comparable across models or papers.
- `eval_loss` < `train_loss` is consistent with *no overfitting* after 1 epoch, but can also be explained by:
  - training loss averaged over high-loss early steps,
  - LoRA dropout on at train, off at eval,
  - eval split possibly slightly easier.
- **Not** evidence of superior generalisation.

Qualitative inspection — 3 held-out test samples

#	Declared smells	Human ground truth	Fine-tuned model output
1	`apt_no_install_recommends`, `unpinned_base_image` (Docker)	Version bump ATMOS 1.210 → 1.214 — *does not address the declared smell*	Adds `-allow-downgrades`, creates non-root user `appuser` — sensible but unrelated hardening
2	`uri_credentials` (Terraform)	Changes `recovery_window_in_days` 0 → 7 — unrelated	Adds IRSA / IAM scaffolding, keeps 0 — does not target URI credentials
3	`unpinned_base_image` (Docker)	Pins a Go image by `sha256:` digest (✓ real pin)	Does not pin; performs unrelated hardening (`apt-get upgrade`, labels)

What we see (with $n = 3$ caveat)

- Output is syntactically well-formed on these 3 cases.
- Contains idiomatic hardening patterns.

What we see (worrying)

- Model *and human ground truth* often fail to address the declared smell.
- **The label itself is noisy.**

Label noise is the binding constraint

The problem

Commit-scraped pairs are at **file granularity**, but a single commit may bundle several unrelated modifications.

- Regex classifier infers the smell from the *commit message* (not the diff).
- Scanner validation only confirms that `code_before` was vulnerable — not that the *removed lines* address that vulnerability.

Practical consequence

Training signal = average of (a) genuine smell→patch pairs and (b) unrelated refactors that happen to coexist in a "fix: security" commit.
The model can only learn the distribution of *"things that look like fix commits"*.

Two remediations (for next iteration)

1. **Hunk-level filter**: keep only records whose *diff* regex-matches the declared smell.
2. **Scanner-fixture-only corpus**: $\sim$1,850 Checkov + KICS test pairs where the smell is, by construction, what the scanner flags.

Limitations and threats to validity

Experimental limitations

- **Small model** (2 B parameters)
- **Subset of the data** (3,227 train)
- **1 epoch**, **1 seed**, **1 prompt template**
- No hyperparameter search
- No downstream (scanner-level) metric
- Qualitative look at only $\mathbf{n = 3}$ samples

Threats to validity

- **External**: dataset skewed toward public Docker/Kubernetes repos.
- **Construct**: "smell" = "pattern detected by Checkov/tfsec/KICS"; excludes business-logic flaws.
- **Internal**: single run, no variance estimate.

What this run *does* demonstrate

The end-to-end pipeline runs within Kaggle's free GPU tier, produces a coherent 25 MB adapter, and surfaces a *measurable* obstacle to improvement: label noise.

Integration path: adapter → Ollama

Three new scripts added to the repo

- `04_merge_adapter.py` — `PeftModel.merge_and_unload()`
- `05_convert_to_gguf.sh` — llama.cpp + Q4_K_M quant
- `06_ollama_create.sh` — Modelfile + `ollama create`

Framework change required: one line

Set `OLLAMA_MODEL=iac-fixer` in `.env`; `FixGenerator` already reads the backend from env vars.

Config B evaluation — the remaining deliverable

Evaluation plan (from Rapport 2, Chapter 6)

Config	Composition	Status
A	Checkov alone	**Measured**. F1=0.132, P=0.084, R=0.304
B	Analyzer + fine-tuned LLM + validator (full pipeline)	**Pending** — this work enables it
C	Config B + self-consistency (N=5) + iterative retrieval	Deferred

Primary metrics (consistent with Rapport 2)

- Patch Validity Rate
- Smell Elimination Rate
- No-New-Issues Rate
- Macro-F1 over the ground-truth smell set

Sequencing

1. Merge adapter locally
2. GGUF + Ollama registration
3. Run `scripts/evaluate.py` on the labelled 8-file dataset
4. Compare Config A vs B

Roadmap beyond Config B

Data

- Hunk-level diff filter
- Scanner-fixture-only corpus
- Extend scraper v2 with hunk-level classifier

Model

- Try Qwen2.5-Coder-3B (non-gated, stronger on code)
- 2–3 epochs with early stopping
- Small HP grid search over r and lr
- Prompt-template diversity

Loop

- Close the RAG $\leftrightarrow$ validator loop
- Self-training on validator-failed examples
- Multi-seed replication for variance

The present run is one building block. Closing the loop requires the framework-level evaluation, which is the next concrete step.

Take-aways

What we can say

- A complete QLoRA training pipeline for IaC fix generation runs end-to-end on Kaggle's free T4 in $< 2\,\mathrm{h}$.
- A 25 MB adapter is produced from 3,227 scanner-validated training records; losses are stable, no overfitting signal.
- On three inspected samples, outputs are well-formed and contain idiomatic hardening patterns.

What we cannot say yet

- Anything about downstream quality on real scanner-level metrics — the Config B evaluation has not been run.
- Anything statistically meaningful — one seed, one config, three qualitative samples.

The binding constraint

Label noise in commit-scraped pairs. Fixing it (hunk-level filter, scanner fixtures) is the single most promising next step — more so than scaling the model.

Questions ?

Ahmedou Yahye Kheyri — `ahmedouyahya2011@gmail.com`

Supervisor: Prof. Hala Bezine — FSS Sfax

Based on Rapport 7 — `Rapports/drafts/rapport7/rapport7.pdf`